

Addis Ababa Science and Technology University																				
1	College: Electrical and Mechanical Engineering									Department: Software Engineering										
2	Course Name		Principles of Compiler Design																	
	Course Code:		SWEG4104																	
3	Synopsis:		This course is intended to teach the basic principles involved in compiler design. These include lectures and experiments on compiler design, implementation techniques and tools. The course will provide deeper insights into the fundamental aspects of lexical analysis, syntax analysis, semantic analysis, optimization, code generation, symbol table handling, and error handling. Finally students will design and implement a mini language compiler.																	
4	Name(s) of Academic Staff:		Biruk Gebru																	
5	Semester and Year offered:		Semester:		II			Year:		4										
6	Credit Hour:		3																	
7	Prerequisite/ Co-requisite: (if any)		SWEG4101: Formal Language and Automata Theory																	
8	Course Learning Outcome (CLO): At the end of the course the student will be able to:																			
	CLO1																			
	CLO2		Create lexical and grammatical rules for a typical high level programming language.																	
	CLO3		Use compiler construction tools like Lex/Flex to create a lexical analyzer and Yacc/Bison to create a parser.																	
	CLO4		Modify a parser by adding semantic rules to associate meaningful attributes along with grammars while parsing.																	
	CLO5		Design and implement Lexical analyzer and syntax analyzer without using scanner/parser generation tools.																	
	CLO6		Prioritize and apply better techniques and algorithms for code generation, optimization and intermediate compilation process.																	
9	Mapping of the course Learning Outcomes to the program Learning Outcomes, Teaching Methods and Assessment:																			
	Course Learning Outcomes (CLO)		Program Learning Outcomes (PO)																	
			P01	P02	P03	P04	P05	P06	P07	P08	P09	P010	P011	P012	Teaching Methods	Assessment				
																Test	Quiz	Assignment	Project	Lab-report

														L	T	P	O					
	CLO1		√											√		√		√	√			
	CLO2		√											√		√		√		√	√	√
	CLO3					√								√		√		√		√	√	√
	CLO4	√												√		√		√		√	√	√
	CLO5			√										√		√		√	√			
	CLO6			√																		
	Indicate the relevancy between the CLO and PO by ticking “√”on the appropriate relevant box																					
10	Transferable Skills (if applicable) (Skills learned in the course of study which can be useful and utilized in other settings)																					
	1	A skill of writing code for a lexical analyzer (or use Lex), parser (or use Yacc), intermediate code generator and register allocator																				
	2	Improve debugging and ability to work with recursion																				
	3	Get exposure to how Abstract Syntax Trees and Data Flow Graphs are represented internally in a compiler																				
	4	Ability to solve complex computing problems																				
	5	A skill to represent a finite set of representation (DFA or NFA) for an infinite set of data or alphabets in a language																				
11	Distribution of Student Learning Time (SLT)																					
	Course Content Outline								CLO		Teaching and Learning Activities								Total (SLT)			
Guided learning (F2F)											Guided Learning (NF2F)		Independent Learning (NF2F)									
L											T	P	O									

	Chapter 1: Introduction 1.1 Compiler and its various phases 1.2 Cousins of Compiler 1.3 The Grouping of Phases 1.4 Compiler Construction Tools	CL01	2		0.5	1	2	3	6.5
	Chapter 2: Lexical Analysis 2.1 Functions of Lexical Analysis 2.2 Role of the Lexical Analyzer 2.3 Input Buffering 2.4 Specification of tokens 2.5 Recognition of tokens	CL01 CL02, CL03, CL05	4		6.5	0.5	1	2	14
	Quiz								
	Chapter 3: Syntax Analysis 3.1 The Role of the Parser 3.2 Context Free Grammars 3.3 Regular Expressions Vs. Context Free Grammars 3.4 Predictive Parsing 3.5 Operator Precedence Parsing 3.6 LR Parsers	CL01 CL02, CL03, CL05	4		7	0.5	2	2	15.5
	Chapter 4: Syntax Directed Translation 4.1 Syntax directed definitions 4.2 Construction of syntax trees 4.3 Bottom-up evaluation of S-attributed definitions 4.4 L-attributed definitions	CL01, CL02, CL04	3		6	0.5	2	5	15.5

	Chapter 5: Symbol Tables & Type Checking 5.1 Symbol Tables 5.2 Type Systems 5.3 Specification of a simple type checker	CL01, CL02, CL04, CL06	3		4	1	2	2	12
	Chapter 6: Intermediate Code Generation 6.1 Intermediate Languages 6.2 Declarations	CL01, CL06	3		3		2	4	12
	Chapter 7: Run Time Environments 7.1 Source Language Issues 7.2 Storage organization 7.3 Storage allocation strategies 7.4 Parameter Passing	CL01, CL06	3				4	6	13
	Chapter 8: Code Generation 8.1 Issues in the design of a code generator 8.2 The Target Machine 8.3 Run-Time Storage Management 8.4 Basic blocks and Flow graphs 8.5 Code generation algorithm	CL01, CL06	3		9		1	2	15
	Chapter 9: Code Optimization 9.1 Principle sources of optimization 9.2 Function preserving transformation 9.3 Common Sub expressions 9.4 Copy propagation	CL01, CL06	3		3			6	12
	Total		22		12	12	13	27	95

	Assessment					
	Continuous Assessment		Percentage Total-50(%)	F2F	NF2F	SLT
	1	Quize	5	0.5		0.5
	2	Assignments	10		4	4
	3	Tests	20	1		1
	4	Project	10	1	6.5	7.5
	5	Lab-report	5	0.5	2.5	3
	Total					16
	Final Exam		Percentage 50 (%)	F2F	NF2F	SLT
	Final Exam		50	3	6	9
Grand Total SLT					120	
	L = Lecture, T = Tutorial, P = Practical, O = Others, F2F = Face to Face, NF2F = Non Face to Face Note: indicates the CLO based on the CLO's numbering in item 9.					
12	Special requirements and resources to deliver the course (e.g. software, computer lab, simulation room ...etc.)	1	Software			
		2	Computer Lab			
13	Text book and reference:	1	Reinhard Wilhelm, Helmut Seidl, Sebastian Hack; Compiler Design: Syntactic and Semantic Analysis, (2013), Springer-Verlag Berlin , Heidelberg			
		2	Alfred V. Aho, Ravi Sethi, Jeffrey D. Ull man. “Compilers Principles, Techniques and Tools”. Pearson Education, 2008.			
		3	Torben Ægidius Mogensen; Introduction to Compiler Design, (2011), Springer-Verlag London			
		4	Helmut Seidl, Reinhard Wilhelm, Sebastian Hack; Compiler Design: Analysis and Transformation, (2012), Springer-Verlag Berlin Heidelberg			

		5	Singh R., Sharma V., Varshney M.; Design and implementation of compiler, (2010), New Age International
--	--	---	--